

Universidad Nacional Autónoma de México.



Facultad de Ingeniería.

Sistemas Distribuidos.

Profesor: Ing. Óscar Manuel Ruíz Hurtado.

Semestre 2027-1

Laboratorio S8.

Que la transferencia no deje dinero en el aire.

Grupo: 2

Alumna:

Valencia Lerdo Dalia Jimena

Cd. Universitaria a 17 de octubre de 2026

1. **El problema.**

Antes de modificar el código, probé la transferencia con una pausa de 20 segundos entre el cargo y el abono. Consulté los saldos de las cuentas 369369369 y 963963963, que eran 1000 y 500. Envié una transferencia de 200 y durante la pausa apagué el servicio cuenta. Cuando lo volví a encender, los saldos eran 800 y 500, lo anterior indica que el dinero salió del origen y no llegó al destino.

Imagen 1. Saldos iniciales de las cuentas 369369369 y 963963963 (1000 y 500).

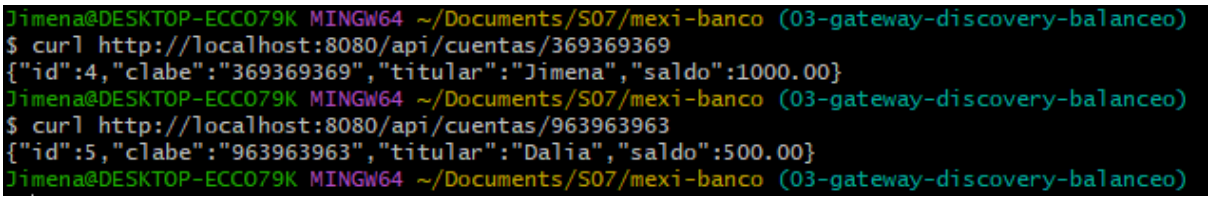


Imagen 2. Reinicio del servicio de transferencia con una pausa de 20 segundos entre el cargo y el abono.

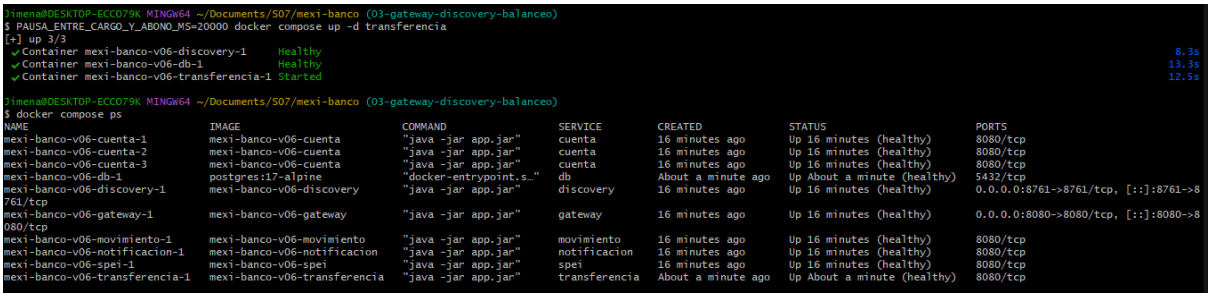
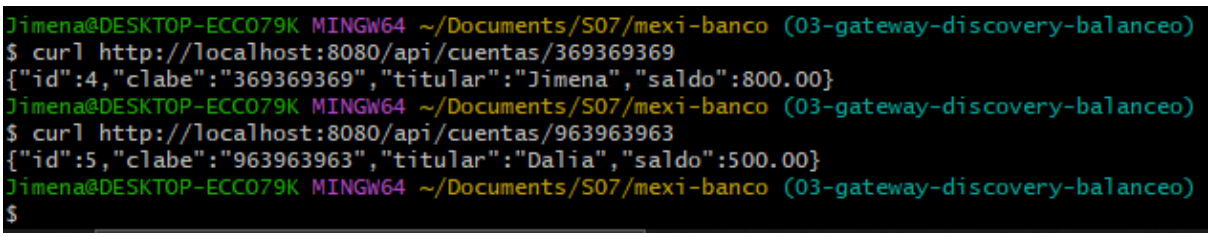


Imagen 3. Saldos después de apagar la cuenta durante la pausa (800 y 500)



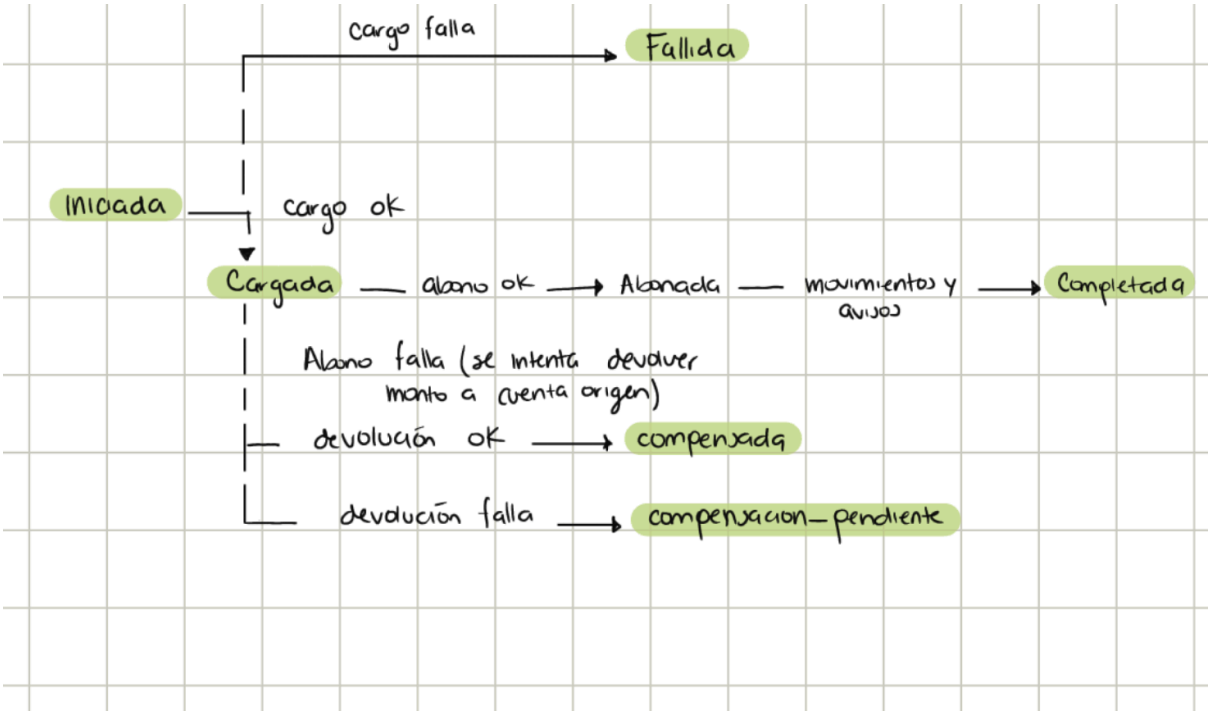
Pasó porque el cargo y el abono son dos llamadas HTTP independientes a un servicio con su propia base de datos, y cada una se confirma sola. No existe un rollback que abarque a los dos servicios, por lo que cuando el abono falló, nada deshizo el cargo. Por otro lado, el código original guardaba la transferencia hasta el final, así que de esa operación fallida no quedó ningún registro.

2. **La saga.**

TransferenciaService funciona como orquestador. Primero guarda la transferencia con estado INICIADA y después ejecuta los pasos, actualizando el estado en la base después de cada uno.

Paso	Si sale bien	Si falla	Compensación
Se carga al origen	CARGADA	FALLIDA	No hace falta, el dinero no salió
Abono a la cuenta destino	ABONADA	Se compensa	Devuelve el monto a cuenta origen
Movimientos y avisos	COMPLETADA		
Devolución a cuenta origen	COMPENSADA	COMPENSACION_PENDIENTE	Reintenta después

Imagen 4. Diagrama de estados de la transferencia.



3. El código clave.

Modifique cinco cosas dentro del código

a) Estado en la transferencia. Creé el enum EstadoTransferencia (INICIADA, CARGADA, ABONADA, COMPLETADA, FALLIDA, COMPENSADA, COMPENSACION_PENDIENTE) y agregué a Transferencia los campos estado y detalle, para que cada transferencia diga en qué paso va y qué pasó si algo falló.

Imagen 5. Campos estado y detalle en la entidad Transferencia.

```

    @Enumerated(EnumType.STRING)
    @Column(length = 30)
    private EstadoTransferencia estado;

    @Column(length = 300)
    private String detalle;
```

b) El orquestador. transferir() guarda la transferencia como INICIADA antes de mover dinero y la vuelve a guardar después de cada paso. Si el cargo falla queda FALLIDA; si el abono falla, llama a compensar().

Imagen 6. Abono dentro de try, con la compensación si falla.

```
CuentaRemota destino;
                                try {
                            fallaDeDemo(claveDestino);
                            destino = cuentas.abonar(claveDestino, monto);
                    } catch (RuntimeException falloAbono) {
                            return compensar(transferencia, falloAbono);
                    }
```

c) La compensación. compensar() devuelve el monto a la cuenta de origen y registra un movimiento. Si lo logra queda COMPENSADA; si la devolución también falla, queda COMPENSACION_PENDIENTE.

Imagen 7. Método compensar().

```
private Transferencia compensar(Transferencia transferencia, RuntimeException causa) {
                try {
                        CuentaRemota devuelta = cuentas.abonar(transferencia.getClaveOrigen(), transferencia.getMonto());
                        movimientos.registrar(devuelta.clabe(), TipoMovimiento.TRANSFERENCIA_RECIBIDA,
                                transferencia.getMonto(), devuelta.saldo(),
                                "devolucion de la transferencia " + transferencia.getId());
                        transferencia.setEstado(EstadoTransferencia.COMPENSADA);
                        transferencia.setDetalle(recortar("El abono a " + transferencia.getClaveDestino()
                                + " falló (" + causa.getMessage() + "). Se devolvió el monto a la cuenta origen."));
                } catch (RuntimeException falloCompensacion) {
                        transferencia.setEstado(EstadoTransferencia.COMPENSACION_PENDIENTE);
                        transferencia.setDetalle(recortar("El abono falló y la devolución también ("
                                + falloCompensacion.getMessage() + "). El monto está pendiente de devolverse al origen."));
                }
                return transferencias.save(transferencia);
        }
```

d) La falla simulada. Agregué la variable de entorno FALLAR_ABONO_HACIA. Si la CLABE destino es igual a su valor, fallaDeDemo() lanza una excepción justo antes de abonar, y así pruebo la compensación sin apagar cuenta.

Imagen 8. Método fallaDeDemo() para simular la falla del abono.

```java
private void fallaDeDemo(String claveDestino) {
        if (!fallarAbonoHacia.isBlank() && fallarAbonoHacia.equals(claveDestino)) {
                throw new IllegalStateException("Falla simulada del abono hacia " + claveDestino);
        }
}
```

e) El circuit breaker. En ResilienciaConfig configuré Resilience4j y en CuentaCliente envolví las tres llamadas con conCircuito(). Si el circuito está abierto responde un 503 con el mensaje “Circuito abierto”. También agregué un caso para el error “No instances available”, que antes salía como 500.

Imagen 9. Configuración del circuit breaker con Resilience4j.

```
.slidingWindowSize(4)                    // mira las últimas 4 llamadas
.minimumNumberOfCalls(4)                 // no decide antes de tener 4
.failureRateThreshold(50)                // si fallan 50% o más, se abre
.waitDurationInOpenState(Duration.ofSeconds(15)) // tiempo abierto
.permittedNumberOfCallsInHalfOpenState(2)        // pruebas en semiabierto
```

4. La compensación funcionando.

Ejecute transferencia con FALLAR_ABONO_HACIA=963963963, que hace fallar el abono hacia esa cuenta sin apagar cuenta. Mandé una transferencia de 100 de la cuenta 369369369 a la 963963963. La respuesta fue COMPENSADA, con un detalle que explica que el abono falló y que se devolvió el monto al origen. Los saldos quedaron igual que antes (700 y 600), porque el cargo se hizo y luego se devolvió. En la base, la transferencia 5 quedó en COMPENSADA y la 4, que fue una transferencia normal, en COMPLETADA.

Imagen 10. Transferencia en estado COMPENSADA y saldos sin cambio neto (700 y 600).

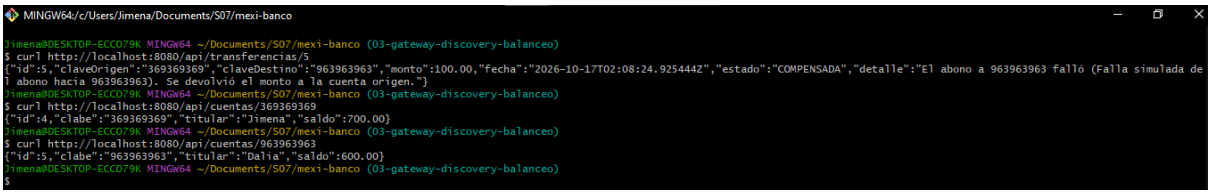
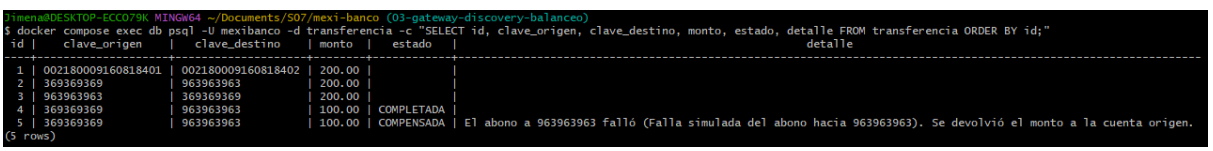


Imagen 11. Tabla transferencia en la base de datos.



Las filas 1, 2 y 3 salen sin estado porque se guardaron antes de agregar esa columna.

5. **Cuando la compensación también falla.**

Repetí la prueba de la Parte 0, pero ahora con mi saga: pausa de 20 segundos y cuenta apagada con docker compose kill cuenta durante la pausa. El gateway respondió 504, pero transferencia siguió trabajando. Al terminar la pausa falló el abono y también la devolución, porque cuenta seguía apagada. La transferencia 6 quedó en COMPENSACION_PENDIENTE, con un detalle que dice que el abono falló y la devolución también (Connect timed out). Los saldos quedaron en 600 y 600: al origen se le quitaron 100 y todavía no se le devuelven.

Imagen 12. Respuesta 504 del gateway mientras cuenta estaba apagada.

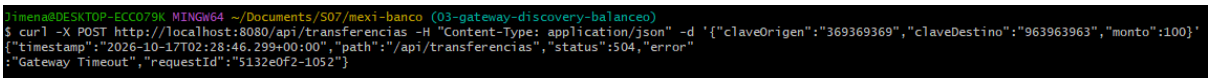


Imagen 13. Transferencia 6 en estado COMPENSACION_PENDIENTE.

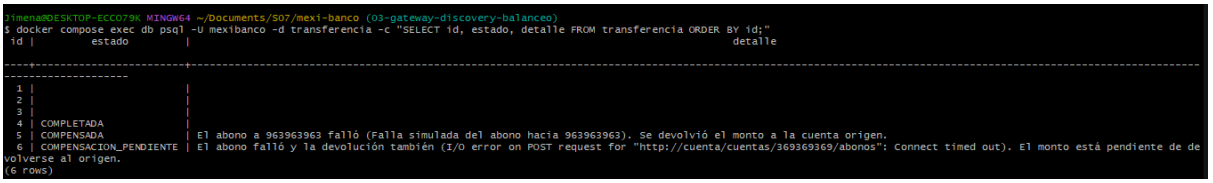
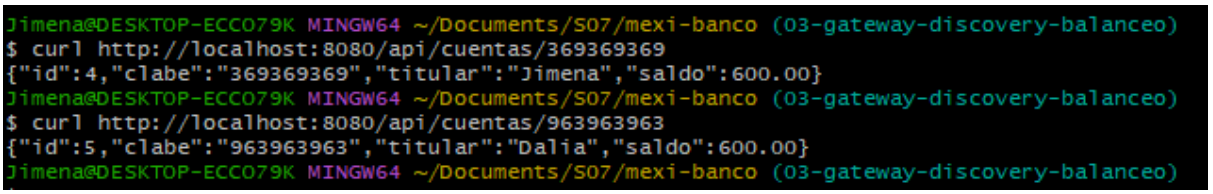


Imagen 14. Saldos finales (600 y 600): el monto sigue pendiente de devolverse.



Para terminarla, haría un proceso programado que busque cada cierto tiempo las transferencias en COMPENSACION_PENDIENTE y reintente la devolución al origen. Cuando cuenta responda, la transferencia pasaría a COMPENSADA; si vuelve a fallar, se reintenta después con esperas cada vez más largas.

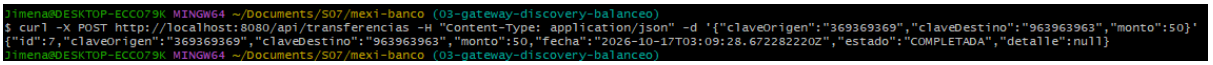
La compensación debe ser idempotente porque un reintento puede ejecutarse dos veces. Por ejemplo, la devolución pudo llegar a cuenta pero la respuesta se perdió y yo creería que falló; si el reintento vuelve a abonar, le devuelvo el dinero dos veces al origen. Idempotente quiere decir que hacerlo una vez o diez veces deja el mismo resultado, y se logra mandando un identificador único de la operación (el id de la transferencia) para que cuenta no aplique el mismo abono dos veces. Mi compensar() actual todavía no es idempotente, y es lo primero que arreglaría antes de agregar reintentos. Como dice la clase, reintentar sin idempotencia cobra dos veces.

6. **El circuit breaker.**

Usé Resilience4j con esta configuración: revisa las últimas 4 llamadas, se abre si fallan 50% o más, se queda abierto 15 segundos y después deja pasar 2 llamadas de prueba.

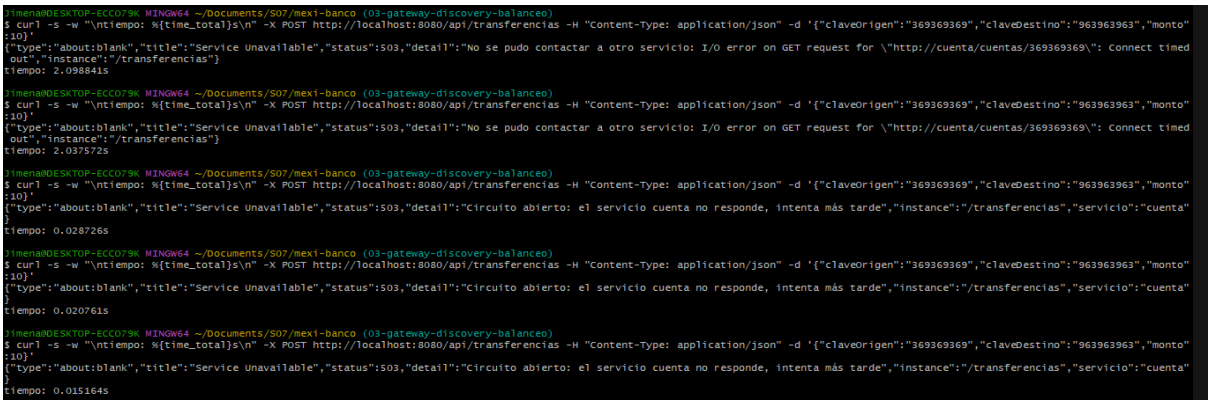
Cerrado. Con cuenta encendida, las transferencias pasan normal y responden COMPLETADA.

Imagen 15. Circuito cerrado: transferencia normal en estado COMPLETADA.



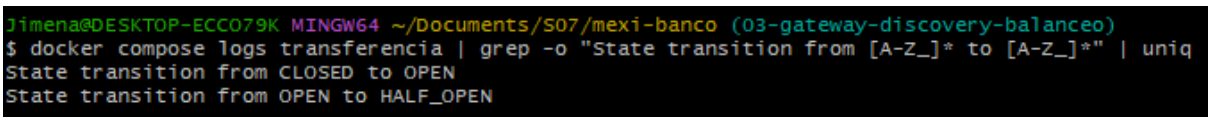
Abierto. Apagué cuenta y mandé varias transferencias. Las dos primeras fallaron con Connect timed out y tardaron unos 2 segundos cada una. Desde la tercera, el circuito ya estaba abierto y respondió en 0.029, 0.021 y 0.015 segundos, con el mensaje “Circuito abierto”, sin llamar a cuenta. Se abrió con solo 2 fallas porque la ventana de 4 llamadas todavía traía llamadas exitosas de antes, y 2 de 4 ya es el 50%.

Imagen 16. Circuito abierto: respuestas rápidas con “Circuito abierto” y sus tiempos



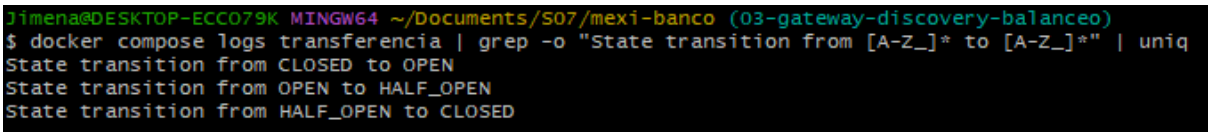
Semiabierto. A los 15 segundos pasó solo de abierto a semiabierto, como se ve en el log.

Imagen 17. Log con los cambios CLOSED a OPEN y OPEN a HALF_OPEN.



Cerrado otra vez. Encendí cuenta, esperé a que Eureka la registrara y mandé una transferencia. Salió bien y el circuito pasó de semiabierto a cerrado.

Imagen 18. Log con la secuencia completa de los tres estados.



En una prueba previa, algunas respuestas salieron como 500 sin explicación. Revisé el log y era el error No instances available for cuenta, que aparece cuando Eureka ya sacó a cuenta de su lista. Lo corregí para que responda un 503 con mensaje.

7. **Lo que la saga no te da.**

La saga no brinda aislamiento. Entre el cargo y la compensación, el dinero sale de la cuenta de origen pero la transferencia todavía puede deshacerse. Si alguien consulta el saldo de origen en ese momento, vería el saldo disminuido, como si la transferencia fuera definitiva, y después el saldo vuelve a subir cuando se compensa. Es un estado intermedio que otros pueden ver, parecido a una lectura sucia. En una transacción normal, nadie vería ese saldo, porque todo se confirma junto o no se confirma. Cada paso de la saga se confirma solo, y la compensación no borra lo que pasó, solo agrega otro movimiento que lo corrige.

Otra limitación que encontré es que si el cargo y el abono salen bien pero falla el registro de movimientos o los avisos, la transferencia se queda en ABONADA y el cliente recibe un error. El dinero sí se movió bien, pero esa transferencia no llega a COMPLETADA. Para solucionarlo haría que esos pasos también se reintenten.